

Informe de Proyecto

Sistema Distribuido de Comercio Electrónico

LibreMercado

Integrantes

Joselyn Montaña

Nicolás Malebrán

Asignatura

Sistemas Distribuidos

Profesor

Juan Torres

15 de junio de 2026

Índice

1	Introducción	2
1.1	Objetivos del Taller	2
1.2	Alcance del proyecto	2
2	Descripción General del Sistema	2
3	Arquitectura del Sistema	3
3.1	Visión general	3
3.2	Datos replicados vs. datos locales	3
3.3	Stack tecnológico	3
4	Requisito 1: Operaciones CRUD	4
4.1	Endpoints implementados	4
4.2	Borrado lógico y consistencia de IDs	4
4.3	Escritura atómica en los tres nodos (MultiNodo)	4
5	Requisito 2: Transacciones ACID	5
5.1	Venta: descuento atómico de stock	5
5.2	Compra: reabastecimiento	6
5.3	Propiedades ACID aplicadas	6
6	Requisito 3: Aplicación del Teorema CAP	6
6.1	Two-Phase Commit manual (traspaso)	6
7	Requisito 4: Backend en PHP	7
7.1	Conexión y configuración	7
7.2	AJAX	7
7.3	Docker	7
8	Interfaz de Usuario	7
8.1	Tienda	7
8.2	Administración	7
9	Pruebas y Validación	8
10	Conclusiones	8
	Anexo: Estructura del Proyecto	9

1. Introducción

El presente informe documenta el desarrollo de **LibreMercado**, un prototipo de sistema distribuido de comercio electrónico desarrollado para el taller de la asignatura Sistemas Distribuidos. El proyecto parte de un borrador inicial de base de datos centralizada ("Libre Mercado") y lo transforma en un sistema con **tres nodos de base de datos independientes**, uno por cada sucursal de la cadena, simulando un entorno real de fragmentación geográfica y alta disponibilidad.

1.1 Objetivos del Taller

- Implementar operaciones CRUD funcionales en un entorno distribuido.
- Garantizar consistencia y transaccionalidad mediante propiedades ACID.
- Analizar y justificar las decisiones arquitectónicas bajo el Teorema CAP.
- Desarrollar la lógica del sistema utilizando el lenguaje PHP.

1.2 Alcance del proyecto

El sistema implementa los cuatro módulos solicitados en el enunciado del taller: gestión de productos, gestión de clientes/usuarios, control de stock por sucursal, procesamiento de ventas mediante un carrito de compras, y reabastecimiento mediante compras a proveedores. Todo el desarrollo se ejecuta sobre contenedores Docker, lo que permite simular tres "servidores" de sucursal de forma aislada y reproducible en una sola máquina.

2. Descripción General del Sistema

LibreMercado está organizado en torno a cinco entidades principales, tal como lo define el enunciado del taller:

- **Productos:** catálogo con `id_producto`, nombre, precio, descripción y categoría.
- **Clientes y Usuarios:** tabla `usuarios` con credenciales, nombre, correo y rol (administrador, operador, cliente).
- **Sucursales y Stock:** tres sucursales (Norte, Centro, Sur), cada una con su propio control de inventario.
- **Carrito y Ventas:** procesamiento de ventas mediante `ventas` y `detalle_ventas`, descontando stock de forma atómica.
- **Compras y Proveedores:** reabastecimiento mediante `compras` y `detalle_compras`, asociadas a un proveedor.

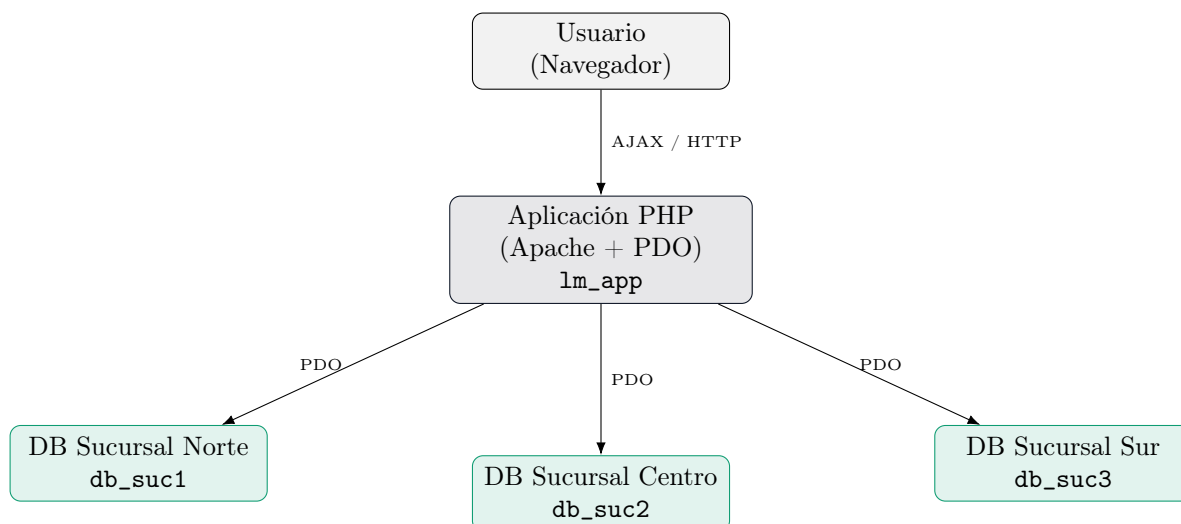
La aplicación se divide en dos interfaces:

- **Tienda** (`index.php`): catálogo consolidado de las tres sucursales, carrito de compras y una pasarela de pago que ejecuta la venta.
- **Administración** (`admin.php`): CRUD de productos, clientes/usuarios y proveedores; reabastecimiento; y trasposos de stock entre sucursales.

3. Arquitectura del Sistema

3.1 Visión general

El sistema está compuesto por **cuatro contenedores Docker**: uno para la aplicación PHP (Apache + PHP 8.2) y tres para los nodos de base de datos (MariaDB 10.4), uno por sucursal. Todos los contenedores comparten una red bridge personalizada (**tienda_net**), lo que permite que la aplicación se conecte a los tres nodos por su nombre de servicio Docker (**db_suc1**, **db_suc2**, **db_suc3**).



3.2 Datos replicados vs. datos locales

Una decisión arquitectónica clave es separar las tablas en dos categorías:

Tipo	Tablas y comportamiento
Catálogo replicado	productos, usuarios, proveedores, sucursales. Existe una copia idéntica (mismo id, mismos datos) en los tres nodos. Toda escritura se aplica a los tres nodos a la vez.
Datos locales por sucursal	stock, ventas, detalle_ventas, compras, detalle_compras. Cada nodo solo contiene la información de <i>su propia</i> sucursal; no se replican entre nodos.

3.3 Stack tecnológico

Capa	Tecnología
Backend	PHP 8.2 + PDO (<code>pdo_mysql</code>)
Base de datos	MariaDB 10.4 (3 instancias independientes)
Servidor web	Apache (imagen <code>php:8.2-apache</code>)
Frontend	HTML5, CSS3 (variables CSS, diseño propio), JavaScript (vanilla)
Comunicación	AJAX (<code>fetch</code> + JSON)
Orquestación	Docker y Docker Compose, red bridge dedicada
Documentación	LaTeX (<code>pdflatex</code>)

4. Requisito 1: Operaciones CRUD

Las operaciones CRUD se implementaron sobre las entidades del catálogo replicado (productos, usuarios/clientes y proveedores), todas con **borrado lógico** mediante un campo **activo**.

4.1 Endpoints implementados

Archivo	Operaciones
api/productos_crud.php	GET listar, POST crear (incluye stock inicial por sucursal), PUT actualizar, DELETE dar de baja / reactivar.
api/clientes_crud.php	GET listar usuarios, POST crear (con contraseña hashada con password_hash), PUT actualizar, DELETE dar de baja / reactivar.
api/proveedores_crud.php	GET listar, POST crear, PUT actualizar, DELETE dar de baja / reactivar.

4.2 Borrado lógico y consistencia de IDs

Ninguna operación de "baja" ejecuta DELETE. En su lugar, se actualiza la columna **activo** a 0. Esto evita romper claves foráneas con **ventas**, **detalle_ventas** o **stock**, y permite "reactivar" un registro más adelante.

Como el catálogo debe ser idéntico en los tres nodos, los **id** de nuevos registros se generan de forma centralizada con `MultiNode::siguienteId()`, que consulta `MAX(id) + 1` en el primer nodo disponible *antes* de insertar el mismo valor en los tres nodos. Así se evita que cada nodo asigne un `AUTO_INCREMENT` distinto para el mismo registro lógico.

4.3 Escritura atómica en los tres nodos (MultiNode)

Toda escritura sobre el catálogo replicado pasa por la clase `MultiNode`, que generaliza el patrón de "todo o nada" a tres nodos:

Listing 1: Fragmento de `MultiNode::ejecutar()`

```
1 public static function ejecutar(callable $accion): array
2 {
3     $conexiones = [];
4
5     // 1) Conectar y abrir transaccion en TODOS los nodos primero.
6     foreach (ConexionNodos::ids() as $idNodo) {
7         try {
8             $pdo = ConexionNodos::get($idNodo);
9             $pdo->beginTransaction();
10            $conexiones[$idNodo] = $pdo;
11        } catch (Throwable $e) {
12            self::rollbackTodos($conexiones);
13            throw new RuntimeException(
14                ConexionNodos::nombre($idNodo) . ' no esta disponible. '
15                . 'Operacion cancelada (sin cambios en ningun nodo).'
16            );
17        }
18    }
19
20    // 2) Ejecutar la accion (INSERT/UPDATE) en cada nodo.
21    try {
22        foreach ($conexiones as $idNodo => $pdo) {
```

```

23         $accion($pdo, $idNodo);
24     }
25 } catch (Throwable $e) {
26     self::rollbackTodos($conexiones);
27     throw new RuntimeException('Operacion cancelada (rollback en los 3
        nodos).');
28 }
29
30 // 3) Confirmar en los 3 nodos.
31 foreach ($conexiones as $pdo) { $pdo->commit(); }
32 return $conexiones;
33 }

```

Si alguno de los tres nodos no responde, la operación se rechaza **antes** de escribir en cualquiera, garantizando que el catálogo nunca quede divergente entre sucursales.

5. Requisito 2: Transacciones ACID

Las operaciones que modifican stock (ventas y compras) son transacciones **locales a un solo nodo**, implementadas con `PDO::beginTransaction()`, `commit()` y `rollBack()`.

5.1 Venta: descuento atómico de stock

El proceso de venta (`api/procesar_venta.php`) realiza, dentro de una misma transacción: verificación de stock con bloqueo de fila (`SELECT ... FOR UPDATE`), descuento de stock, inserción en `ventas` e inserción en `detalle_ventas`. Si el stock es insuficiente o cualquier paso falla, se ejecuta `rollBack()` y **no** queda ningún cambio aplicado.

Listing 2: Fragmento de `procesar_venta.php`

```

1  $pdo->beginTransaction();
2
3  $stmt = $pdo->prepare("SELECT cantidad FROM stock WHERE id_producto = ?
        FOR UPDATE");
4  $stmt->execute([$idProducto]);
5  $stockActual = $stmt->fetchColumn();
6
7  if ($stockActual < $cantidad) {
8      throw new RuntimeException("Stock insuficiente (disponible:
        $stockActual).");
9  }
10
11 $pdo->prepare("UPDATE stock SET cantidad = cantidad - ? WHERE id_producto
        = ?")
12     ->execute([$cantidad, $idProducto]);
13
14 $pdo->prepare("INSERT INTO ventas (id_cliente, id_sucursal, total, fecha,
        estado)
15             VALUES (?, ?, ?, NOW(), 'completada')")
16     ->execute([$idCliente, $idSucursal, $subtotal]);
17 $idVenta = (int)$pdo->lastInsertId();
18
19 $pdo->prepare("INSERT INTO detalle_ventas (id_venta, id_producto, cantidad
        ,
20             precio_unitario, subtotal) VALUES (?, ?, ?, ?, ?)")
21     ->execute([$idVenta, $idProducto, $cantidad, $precio, $subtotal]);
22
23 $pdo->commit();

```

5.2 Compra: reabastecimiento

El proceso de compra (`api/procesar_compra.php`) es análogo, pero en sentido inverso: valida el proveedor, recorre cada ítem de la orden, inserta `compras` y `detalle_compras`, y **aumenta** el stock correspondiente. Cualquier error (proveedor inactivo, producto inexistente, cantidades inválidas) provoca `rollBack()` completo.

5.3 Propiedades ACID aplicadas

- **Atomicidad:** cada venta/compra es indivisible gracias a `beginTransaction()/commit()/rollBack()`.
- **Consistencia:** el `SELECT ... FOR UPDATE` y las validaciones de stock evitan que el stock quede en estado inválido (negativo).
- **Aislamiento:** el bloqueo de fila (`FOR UPDATE`) evita condiciones de carrera si dos ventas concurrentes intentan descontar el mismo producto.
- **Durabilidad:** al hacer `commit()`, MariaDB (motor InnoDB) garantiza la persistencia del cambio.

6. Requisito 3: Aplicación del Teorema CAP

El grupo eligió el modelo **CP (Consistencia + Tolerancia a Particiones)**. El detalle completo de esta decisión, junto con la tabla de comportamiento ante fallos de conexión entre nodos, se desarrolla en el documento de arquitectura adjunto (`arquitectura.pdf`). A modo de resumen:

- **Lecturas:** disponibilidad parcial. Si un nodo está caído, el catálogo de stock muestra "N/D" para esa sucursal, pero el resto de la página sigue funcionando (`api/stock_consolidado.php`).
- **Ventas y compras:** transacción ACID local; si el nodo de la sucursal no responde, la operación se rechaza (HTTP 503) sin afectar a las demás sucursales.
- **Traspasos entre sucursales:** Two-Phase Commit manual entre dos nodos (`api/traspaso_stock.php`). Si origen o destino no responden, o no hay stock suficiente, **ambos** nodos hacen `rollback`.
- **CRUD de catálogo:** `commit` atómico de los tres nodos mediante `MultiNodo`, descrito en la sección anterior.

6.1 Two-Phase Commit manual (traspaso)

Listing 3: Fragmento de `traspaso_stock.php`

```
1 // FASE 1: PREPARE
2 $pdoOrigen->beginTransaction();
3 $pdoDestino->beginTransaction();
4
5 // Verificar y descontar stock en origen
6 $stockOrigen = ...; // SELECT ... FOR UPDATE
7 if ($stockOrigen < $cantidad) { throw new RuntimeException("Stock
    insuficiente."); }
8 $pdoOrigen->prepare("UPDATE stock SET cantidad = cantidad - ? WHERE
    id_producto = ?")
9     ->execute([$cantidad, $idProducto]);
10
11 // Aumentar stock en destino
12 $pdoDestino->prepare("UPDATE stock SET cantidad = cantidad + ? WHERE
    id_producto = ?")
```

```

13         ->execute([$cantidad, $idProducto]);
14
15 // FASE 2: COMMIT (solo si ambas fases 1 fueron exitosas)
16 $pdoOrigen->commit();
17 $pdoDestino->commit();
18 // En caso de error: rollBack() en ambos

```

Esta es la justificación práctica de CP: ante una partición, el sistema **falla de forma segura**, rechazando la operación completa en lugar de aplicarla parcialmente.

7. Requisito 4: Backend en PHP

7.1 Conexión y configuración

La clase `ConexionNodos` centraliza la configuración de los tres nodos (host, base de datos, credenciales) y entrega instancias PDO configuradas con `PDO::ERRMODE_EXCEPTION`, `PDO::FETCH_ASSOC` y un timeout corto (`PDO::ATTR_TIMEOUT = 1`) que permite detectar un nodo caído en aproximadamente un segundo.

7.2 AJAX

Toda la interacción entre el navegador y el backend ocurre mediante **fetch** hacia los endpoints en `api/*.php`, que devuelven JSON. La página nunca se recarga: el catálogo se refresca automáticamente cada 5 segundos (`setInterval`), y cada acción (vender, comprar, traspasar, CRUD) actualiza solo las partes de la interfaz afectadas.

7.3 Docker

El archivo `docker-compose.yml` define los cuatro servicios (`app`, `db_suc1`, `db_suc2`, `db_suc3`) sobre la red `tienda_net`. Cada base de datos se inicializa con su propio script SQL (`sql/suc1.sql`, `sql/suc2.sql`, `sql/suc3.sql`), generado a partir del export original de phpMyAdmin, separando el stock y las ventas/compras por sucursal y replicando el catálogo en los tres.

8. Interfaz de Usuario

8.1 Tienda

La página principal muestra el estado de los tres nodos en el encabezado (indicador verde/rojo), un catálogo con buscador y filtros por categoría, y por cada producto un chip de stock por sucursal (Norte / Centro / Sur), coloreado según disponibilidad. El carrito permite elegir desde qué sucursal se descuenta cada producto. Al pagar, se abre una pasarela de pago de pantalla completa que muestra el resumen, solicita datos de facturación, y ejecuta la venta mostrando el o los nodos afectados.

8.2 Administración

El panel de administración organiza las operaciones de back-office en pestañas:

- **Productos, Clientes/Usuarios, Proveedores:** CRUD con tabla, modal de creación/edición y botones de baja/reactivación.
- **Reabastecimiento:** armado de una orden de compra a un proveedor, con historial de compras por sucursal.
- **Traspasos entre Sucursales:** formulario de traspaso 2PC entre dos nodos.

9. Pruebas y Validación

Se realizaron pruebas manuales simulando fallos de red mediante `docker stop` sobre el contenedor de un nodo (por ejemplo, `lm_db_suc2`):

- **Lectura con nodo caído:** el catálogo sigue mostrando los productos; la columna de la sucursal afectada muestra "N/D" y su indicador en el encabezado cambia a rojo en menos de 5 segundos.
- **Venta en sucursal caída:** se rechaza con un mensaje explícito, sin afectar el resto del carrito ni otras sucursales.
- **Traspaso con un nodo caído:** se cancela el traspaso completo, sin descuentos ni acreditaciones parciales.
- **CRUD de catálogo con un nodo caído:** la creación/edición/baja se rechaza antes de escribir en los nodos sanos, preservando la consistencia del catálogo.
- **Stock insuficiente:** tanto en venta como en traspaso, se rechaza la operación indicando el stock disponible, sin modificar la base de datos.

Al reactivar el nodo (`docker start`), el sistema vuelve a operar con normalidad sin intervención manual adicional.

10. Conclusiones

El desarrollo de LibreMercado permitió aplicar de forma concreta los cuatro objetivos del taller: se implementó un CRUD completo con borrado lógico sobre un catálogo replicado en tres nodos; se garantizaron las propiedades ACID mediante transacciones PDO locales para ventas y compras; se justificó y demostró en la práctica una arquitectura **CP**, combinando transacciones ACID locales, Two-Phase Commit manual entre dos nodos, y un commit atómico de tres nodos para el catálogo; y todo el backend se desarrolló en PHP con PDO, expuesto mediante AJAX y orquestado con Docker Compose.

La principal lección del proyecto es que, en un sistema distribuido real, **el alcance de la transacción determina el mecanismo**: una operación que toca un solo nodo se resuelve con transacciones PDO estándar; una que toca dos nodos requiere 2PC; y una que debe mantener idéntico un catálogo en N nodos requiere un commit atómico generalizado. En todos los casos, ante una partición de red, el sistema prioriza **no aplicar cambios parciales** por sobre seguir respondiendo, lo cual es coherente con el dominio de un sistema de inventario donde la inconsistencia tiene un costo de negocio directo.

Anexo: Estructura del Proyecto

Listing 4: Estructura de carpetas

```
1 libre_mercado/
2 |-- docker-compose.yml
3 |-- sql/                (esquema + datos iniciales por nodo)
4 |   |-- suc1.sql, suc2.sql, suc3.sql
5 |-- docs/               (documentos de arquitectura e informe)
6 |-- app/
7 |   |-- Dockerfile
8 |   |-- src/
9 |       |-- index.php      (tienda)
10 |       |-- admin.php     (administracion)
11 |       |-- ConexionNodos.php (conexiones PDO a los 3 nodos)
12 |       |-- MultiNodo.php  (commit atomico de 3 nodos)
13 |       |-- assets/        (CSS y JS)
14 |       |-- api/
15 |           |-- stock_consolidado.php
16 |           |-- procesar_venta.php
17 |           |-- procesar_compra.php
18 |           |-- traspaso_stock.php
19 |           |-- productos_crud.php
20 |           |-- clientes_crud.php
21 |           |-- proveedores_crud.php
22 |           |-- compras_listar.php
```

Para levantar el sistema: `docker compose up -d --build` y acceder a <http://localhost:8080>. El panel de administración está disponible en `/admin.php`.